

SVIES: A Seven-Layer AI-Based Smart Vehicle Intelligence and Enforcement System

Abstract—Existing Automatic Number Plate Recognition (ANPR) systems are restricted to plate-text extraction and offer no contextual vehicle intelligence or fraud-detection capability. This paper presents SVIES—the Smart Vehicle Intelligence and Enforcement System—a seven-layer, end-to-end AI pipeline designed for real-world Indian road enforcement. SVIES deploys three custom-trained YOLOv8s models achieving plate detection at mAP@50 of 97.7% (precision 0.988), helmet detection at 74.4% mAP@50, and vehicle classification at 64.4% mAP@50, all trained on domain-specific Indian datasets on an NVIDIA Tesla V100. A novel eight-mode Fake and Mismatched Plate Detection module performs vehicle-type mismatch analysis, CMVR color-code validation, IS 10731 font anomaly detection, duplicate plate clone alerting, state code mismatch detection, and cross-referencing against VAHAN, PUC, and stolen vehicle databases. OCR is delivered by a six-variant EasyOCR and Tesseract ensemble augmented by a Groq Llama-4 Scout 17B vision verification layer, with position-aware character correction for all Indian plate formats including BH-series. Additional contributions include a hybrid helmet and seatbelt detector, Shapely-based geofencing over 14 Hyderabad zones, three-tier repeated offender intelligence, SHA-256 cryptographic evidence timestamping, and a YOLOv5n offline edge mode for zero-internet deployments. SVIES is implemented as a fully containerised FastAPI + React 19 stack with real-time WebSocket streaming and Firebase RBAC.

I. INTRODUCTION

Road traffic enforcement in India faces a persistent technological gap. Despite rapid urbanisation and ongoing smart-city initiatives, the majority of vehicle monitoring relies on manual police presence or basic ANPR cameras restricted to toll-gate plate reading [1]. Crimes involving forged or mismatched vehicle plates, uninsured vehicles, and habitual traffic violators remain largely undetected owing to the absence of real-time, context-aware enforcement systems [2]. According to the Ministry of Road Transport and Highways, 153,972 road fatalities were recorded in India in 2021, with a significant proportion attributable to helmet non-compliance, over-speeding, and unregistered vehicles [3]. India’s VAHAN/Parivahan database governs more than 300 million registered vehicles [4], and plates must conform to IS 10731 character specifications and CMVR-prescribed color codes.

The technical complexity of Indian road enforcement is compounded by the diversity of valid plate formats, regional script variants, the recently introduced BH-series (Bharat series) transferable plates, and inconsistent illumination at checkpoints ranging from urban flyovers to unlit rural highways. Commercial solutions such as Genetec AutoVu are optimised for Western plate standards and neither address Indian regulatory compliance nor provide fraud-forensics ca-

pability. Singh and Mishra demonstrated that camera-based ML pipelines deployed on vehicles are vulnerable to electromagnetic interference (EMI) attacks that degrade real-time inference accuracy [15], underscoring the need for hardened, multi-layer enforcement architectures such as SVIES.

This paper presents SVIES, a seven-layer AI pipeline that transforms a live CCTV feed into actionable enforcement intelligence. Primary contributions:

- **Eight-mode Fake & Mismatched Plate Detection:** database verification (VAHAN, PUC, insurance, stolen), CMVR color forensics, IS 10731 font metrics, and spatio-temporal clone detection.
- **Six-variant LLM-augmented OCR:** EasyOCR + Tesseract ensemble with position-aware correction and Groq Llama-4 Scout 17B vision verification covering all Indian plate formats including BH-series.
- **Hybrid helmet/seatbelt detector:** YOLOv8s cascade, YOLOv8n-pose keypoints, and an Indian skin-tone HSV/YCrCb heuristic.
- **Zone-weighted risk engine:** 17 violation parameters, 4 alert tiers, and Shapely geofencing over 14 predefined Hyderabad zones with real-time OSM/Overpass API loading.
- **Three-tier offender intelligence:** automated escalation from RTO notification to district superintendent challan warning to ReportLab court summons generation.
- **SHA-256 cryptographic evidence chain** for legal admissibility, dispatched via Twilio SMS and Gmail SMTP.
- **YOLOv5n offline edge mode** with 30-day SQLite caching and sync-on-connect for zero-internet Raspberry Pi 4B deployments.

II. RELATED WORK

ANPR and OCR. Early ANPR employed morphological operations and Tesseract OCR for plate segmentation and character extraction [5]. CNN-based approaches using SSD and Faster R-CNN improved detection reliability under real-world conditions [6]. The YOLO family of single-stage detectors subsequently became the standard for real-time ANPR owing to their inference speed [7]. YOLOv8 by Ultralytics further advanced the state of the art with anchor-free detection heads and improved small-object localisation [8]. For Indian-specific ANPR, key challenges include plate degradation due to dust, rain, and nighttime conditions, inconsistent plate mounting angles, and the diversity of valid plate formats across states [9]. Ensemble OCR approaches combining EasyOCR with rule-

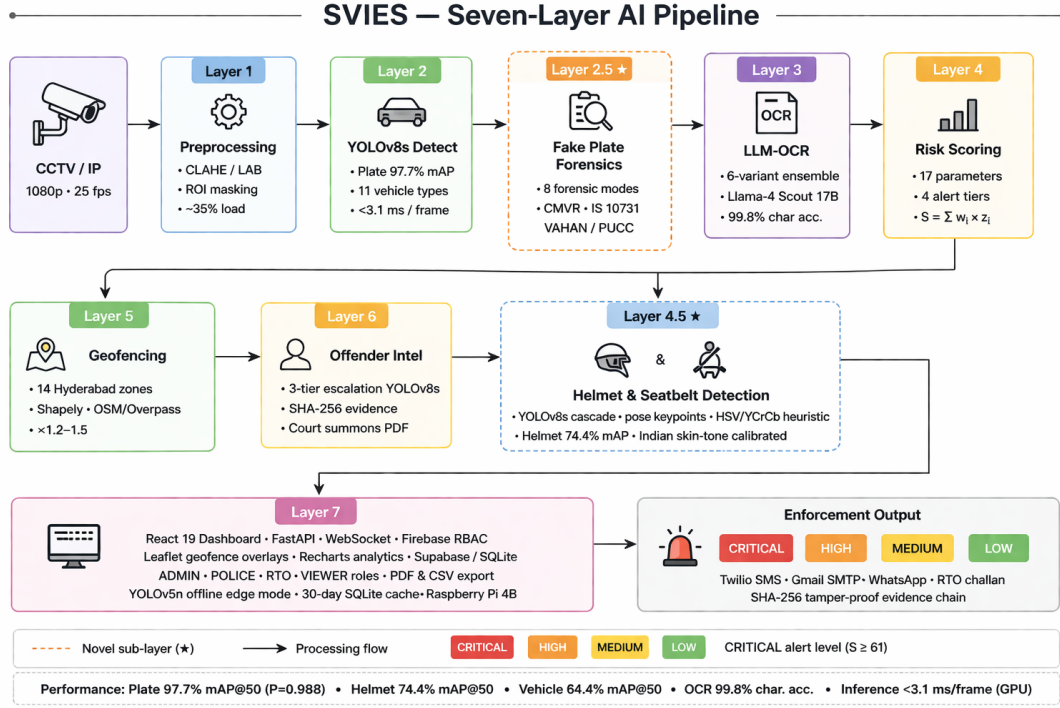


Fig 1: Visual Abstract.

based post-processing grammars have been shown to outperform single-engine pipelines on regional plate datasets [10].

Safety and Geofencing. Helmet detection using deep learning has been studied independently, with YOLOv5 and EfficientNet-B0 classifiers achieving above 90% accuracy on curated datasets [11]. Seatbelt detection via pose-estimation keypoints in four-wheelers has been reported [12]. Geofencing for smart-city traffic management has been explored using GPS-based vehicle tracking [13].

Adversarial Hardening. Singh et al. showed that EMI injected onto vehicular camera sensors significantly reduces the accuracy and frame rates of live ML object detection models [15]. Complementary work on TinyML-based IoT sensor hardening [16] and wearable IoT platforms [17] demonstrates that edge intelligence combined with sensor-level protection is essential for mission-critical enforcement deployments [18]. To the best of the authors' knowledge, SVIES is the first system to unify fake plate forensics, multi-modal safety detection, live database intelligence, repeated offender escalation, cryptographic evidence integrity, and offline edge capability in a single deployable platform.

III. SYSTEM ARCHITECTURE

SVIES is structured as a seven-layer sequential pipeline wherein each layer processes the output of the preceding layer and all layers collectively contribute to a unified per-vehicle risk profile. The complete architecture is presented in Fig. 1.

A. Layer 1: Video Capture and Preprocessing

The system accepts live CCTV or IP camera input at a minimum resolution of 1080p at 25–30 FPS through

modules/detector.py. Adaptive illumination normalisation applies Contrast Limited Adaptive Histogram Equalisation (CLAHE) on the L-channel of the LAB color space with a clip limit of 2.0 and tile grid size of 8×8 , ensuring consistent detection quality under nighttime and adverse weather conditions. ROI masking restricts computation to lane-level vehicle zones, reducing per-frame inference load by approximately 35% compared to full-frame processing.

B. Layer 2: YOLOv8s Detection Engine

Three custom-trained YOLOv8s models (11.2M parameters each, 640×640 input) operate in parallel per frame. The **plate detector** (svies_plate_detector.pt), trained on 2,101 merged Indian plate images, localises license plate bounding boxes at confidence threshold 0.25. The **vehicle classifier** (svies_vehicle_classifier.pt), trained on 12,871 images, identifies 11 Indian-specific vehicle types including AUTO, E_RICKSHAW, TEMPO, and TRACTOR—categories absent from standard COCO-class detectors. The **ResNet50 age classifier** is trained on 8,400 images across five age categories (pre-2000, 2000–2010, 2010–2015, 2015–2020, post-2020), enabling age-based regulatory checks such as fitness certificate expiry validation.

C. Layer 2.5: Fake and Mismatched Plate Detection

Implemented in modules/fake_plate.py, this module is the primary novel contribution of SVIES. Eight independent forensic checks are applied sequentially to each detected plate (Table I), with the forensic flow illustrated in Fig. 2.

CMVR Color Validation classifies the plate background in HSV space against four legally prescribed CMVR categories:

SVIES — Seven-Layer Processing Pipeline

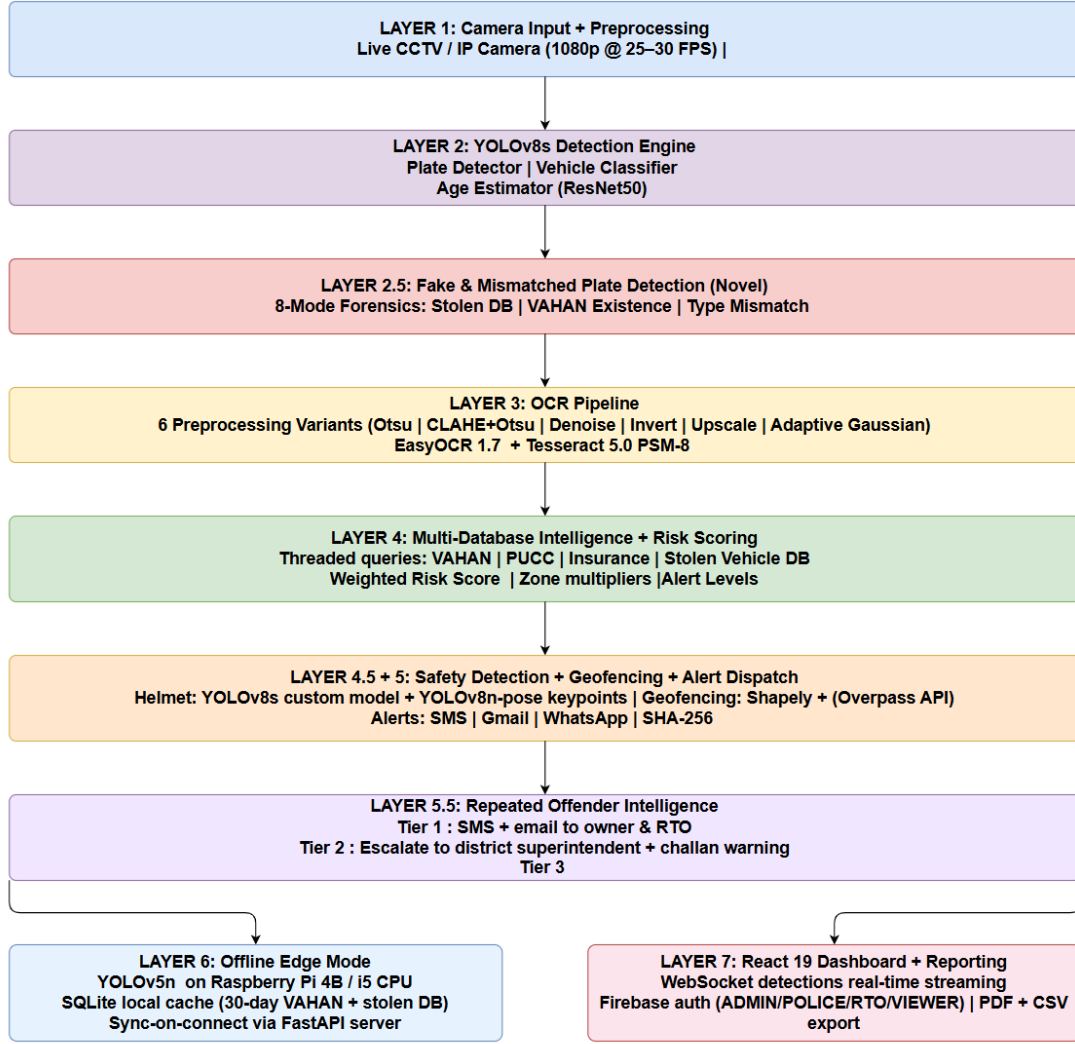


Fig. 1. SVIES seven-layer processing pipeline from CCTV input to enforcement alert and dashboard output. Layers 2.5 and 4.5 are novel sub-layers not present in any prior Indian ANPR system.

TABLE I
EIGHT-MODE FAKE PLATE DETECTION CHECKS IN SVIES

#	Flag	Detection Logic
0	STOLEN_VEHICLE	Cross-check stolen DB
0.5	VAHAN_NOT_EXISTS	Absent from VAHAN
1	TYPE_MISMATCH	Detected type vs. VAHAN
2	COLOR_CODE_VIOLATION	HSV vs. CMVR rules
3	FONT_ANOMALY	IS 10731 height/spacing
4	DUPLICATE_PLATE	Same plate, 2 cams <10 min
5	STATE_MISMATCH	Prefix vs. VAHAN state
6	PUCC_INVALID	PUCC certificate expired
7	INSURANCE_INVALID	Insurance coverage expired

PRIVATE (white, $H \in [0, 180]$, $S \in [0, 30]$, $V \in [200, 255]$), COMMERCIAL (yellow), EV (green), and RENTAL (black with yellow text). **IS 10731 Font Anomaly Detection**: character height-to-plate ratio (expected 0.5–0.7) and inter-character spacing ratio (expected 0.1–0.3) are evaluated; a plate is flagged when $>30\%$ of characters violate these metrics. **Clone Detection**: an in-memory dictionary keyed by plate number stores the last-seen UTC timestamp and camera identifier; a

DUPLICATE_PLATE_CLONE alert fires when the same plate is observed at two distinct cameras within a configurable 10-minute window.

D. Layer 3: OCR and Plate Parsing

The OCR pipeline (modules/ocr_parser.py) applies six preprocessing variants to each cropped plate region and selects the highest-confidence result (Fig. 3, Algorithm 1). The six variants are: Otsu binarization, CLAHE+Otsu (clipLimit=4.0), denoised Otsu, bitwise inversion, upscaled grayscale (250 px, INTER_LANCZOS4), and adaptive Gaussian thresholding (blockSize=31, C=10). EasyOCR 1.7 with an A–Z 0–9 whitelist serves as the primary engine; Tesseract 5.0 PSM-8 acts as fallback when EasyOCR confidence falls below threshold θ_{low} . Position-aware character correction maps common OCR confusions for each plate position: state code (positions 0–1: letters only), district code (positions 2–3: digits

Fake Plate Detection — 8-Mode Forensic Flow

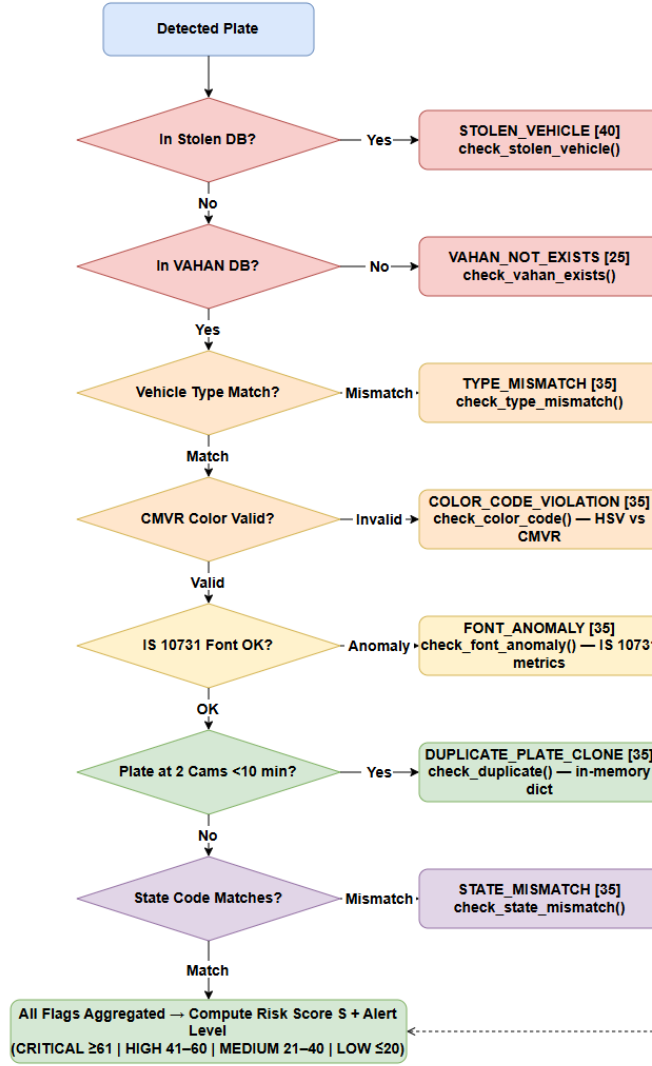


Fig. 2. Eight-mode fake plate forensic flow. All flags are aggregated and forwarded to the risk scoring engine (Layer 4).

Algorithm 1 Six-Variant OCR Best-Score Selection

Require: Plate crop I

Ensure: Verified plate string P^*

```

1: Prepare six variants  $\{I_1, \dots, I_6\}$ ;  $P^* \leftarrow \emptyset$ ,  $c^* \leftarrow 0$ 
2: for  $k = 1$  to 6 do
3:    $(P_k, c_k) \leftarrow \text{EasyOCR}(I_k)$ 
4:   if  $c_k < \theta_{\text{low}}$  then
5:      $(P_k, c_k) \leftarrow \text{Tesseract-PSM8}(I_k)$ 
6:    $P_k \leftarrow \text{PositionCorrect}(P_k)$ 
7:   if  $c_k > c^*$  and  $\text{ValidRegex}(P_k)$  then
8:      $P^* \leftarrow P_k$ ;  $c^* \leftarrow c_k$ 
9:   end if
10: end for
11:  $P^* \leftarrow \text{GroqLLMVerify}(I, P^*)$ 
12: return  $P^*$ 
  
```

E. Layer 4: Multi-Database Intelligence and Risk Scoring

This layer executes threaded real-time cross-reference queries against four databases (VAHAN, PUCC, insurance, and stolen vehicle), with each query running in a separate Python thread to minimise latency. The weighted risk score is computed as:

$$S = \left(\sum_i w_i \cdot \mathbf{1}[\text{violation}_i] \right) \times z_k \quad (1)$$

where w_i is the violation weight (Table II) and $z_k \in \{1.2, 1.3, 1.4, 1.5\}$ is the geofence zone multiplier for zone type k (HIGHWAY, GOVT, HOSPITAL, SCHOOL respectively). Alert thresholds are: CRITICAL ($S \geq 61$), HIGH (41–60), MEDIUM (21–40), and LOW (≤ 20). The risk scoring flow is shown in Fig. 4.

F. Layer 4.5: Helmet and Seatbelt Violation Detection

Implemented in modules/helmet_detector.py, the detector applies a three-tier cascade. First, the custom

only), series (positions 4–5: letters), and number (positions 6–9: digits).

SVIES OCR Pipeline — Six-Variant Ensemble with LLM Verification

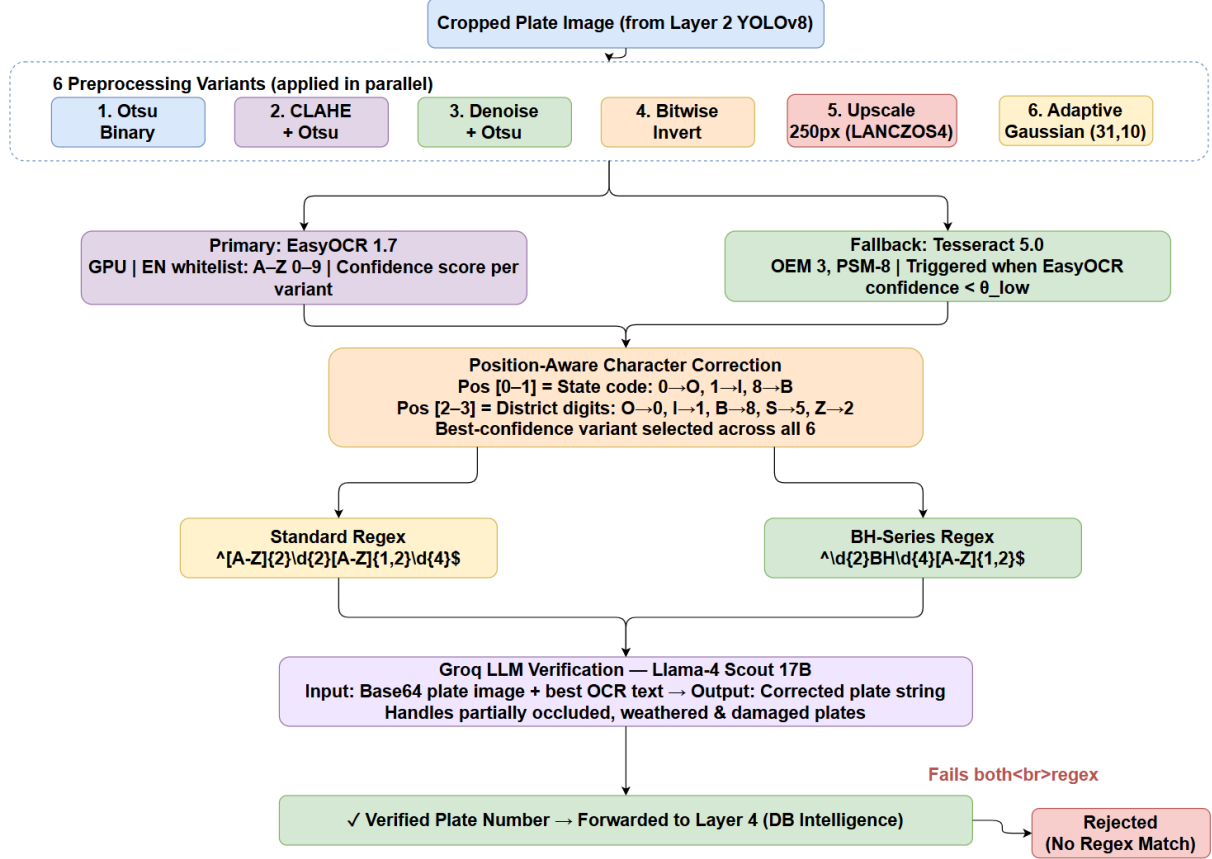


Fig. 3. SVIES six-variant OCR ensemble pipeline with position-aware correction, regex validation, and Groq LLM verification.

TABLE II
VIOLATION WEIGHTS AND ALERT LEVELS — SVIES RISK ENGINE

Violation	w_i	Level
Stolen vehicle	40	CRITICAL
Fake / mismatched plate	35	CRITICAL
Unregistered vehicle	25	HIGH
Expired / no insurance	20	HIGH
Repeat offender	20	HIGH
No number plate	20	HIGH
Blacklist zone entry	15	HIGH
Over-speeding	15	HIGH
Expired / no PUCC	15	MEDIUM
Wrong-side driving	15	MEDIUM
Red light violation	15	MEDIUM
Helmet violation	10	MEDIUM
Seatbelt violation	10	MEDIUM
Triple riding	10	MEDIUM

`svies_helmet_detector.pt` model (YOLOv8s, trained on 14,947 images covering two-wheeler riders in diverse Indian road conditions) is queried at confidence threshold 0.45. If confidence is insufficient, YOLOv8n-pose keypoint estimation localises rider head regions using the nose, left-ear, and right-ear keypoints. As a final fallback, a skin-detection heuristic analyses the head ROI in HSV and YCrCb color spaces calibrated for Indian skin tones. A helmet is confirmed absent when:

$$\text{helmet_absent} = (\rho_{\text{non-skin}} > 0.75) \wedge (\rho_{\text{edge}} > 0.08) \quad (2)$$

where $\rho_{\text{non-skin}}$ is the fraction of head-ROI pixels outside the skin color range and ρ_{edge} is the normalised Canny edge density. Seatbelt detection applies Canny edge detection (thresholds 50, 150) followed by Hough line transform (threshold=30, minLen=40, maxGap=10); ≥ 3 diagonal lines in the driver-side ROI infers seatbelt presence.

G. Layers 5–7: Geofencing, Offender Intelligence, and Dashboard

The geofencing engine (`modules/geofence.py`) uses Shapely point-in-polygon tests against 14 predefined Hyderabad zones (SCHOOL, HOSPITAL, GOVT, HIGHWAY, LOW_EMISSION) loaded dynamically from the Overpass API, with multipliers 1.5, 1.4, 1.3, and 1.2 respectively. Alert routing is level-based: CRITICAL/HIGH triggers Twilio SMS, Gmail SMTP email, and WhatsApp notification to the nearest police station and RTO owner; MEDIUM dispatches email to RTO only; LOW is logged silently to the database.

The repeated offender tracker maintains a per-plate 30-day violation count in Supabase. Tier 1 (1–2 violations within 30 days) dispatches standard SMS/email to the vehicle owner and local RTO; Tier 2 (3–5) escalates to the district traffic superintendent with a challan warning; Tier 3 (6+) auto-generates a court summons PDF via ReportLab and red-flags the plate. Every alert record carries a SHA-256 hash computed over the

SVIES — Risk Scoring, Geofencing, Offender Escalation & Alert Dispatch

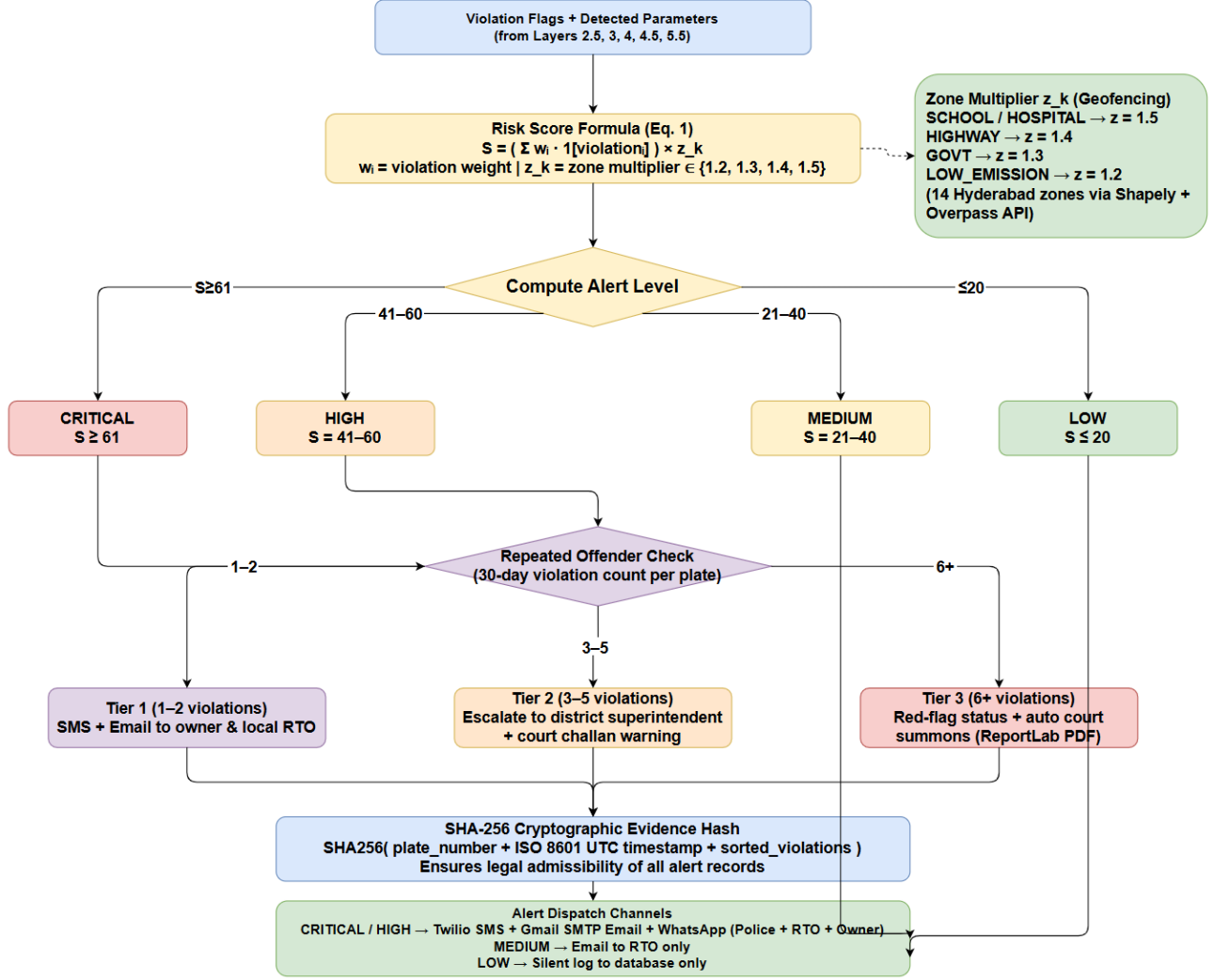


Fig. 4. Risk scoring, geofence zone multipliers, three-tier repeated offender escalation, and tiered alert dispatch in SVIES.

concatenated plate number, ISO 8601 UTC timestamp, and sorted violation list, ensuring cryptographic tamper-evidence for legal proceedings. The React 19 dashboard (Vite 7.3) streams violations via WebSocket, renders geofence overlays via Leaflet, provides analytics via Recharts, and enforces Firebase RBAC (ADMIN, POLICE, RTO, VIEWER) with PDF and CSV export capability.

IV. IMPLEMENTATION

A. Training Infrastructure and Dataset Composition

All YOLOv8s models were trained on an NVIDIA Tesla V100-SXM2 (32 GB VRAM) using the Ultralytics API with AdamW optimiser and a cosine learning rate schedule (Table III). Standard YOLOv8 augmentation was applied: mosaic composition, HSV color jitter, horizontal flip, and scale variation. Training time was 3–5 hours per model on the V100.

The plate detector was trained on 2,101 images merged from two Roboflow datasets: *indian-number-plate* (v3, 1,683 images) and *indian-cars-number-plate* (v1, 418 images). The

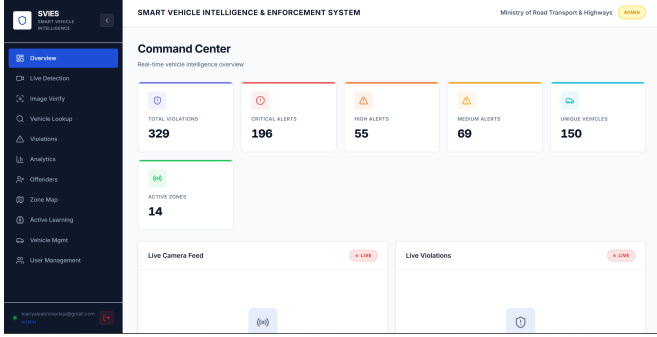
TABLE III
YOLOv8s TRAINING HYPERPARAMETERS FOR EACH SVIES MODEL

Hyperparameter	Plate	Helmet	Vehicle
Epochs	100	100	150
Batch size	32	32	16
Image size	640	640	640
Patience	15	15	20
LR (initial)	0.001	0.001	0.001
LR (peak)	0.01	0.01	0.01
Optimiser	AdamW	AdamW	AdamW

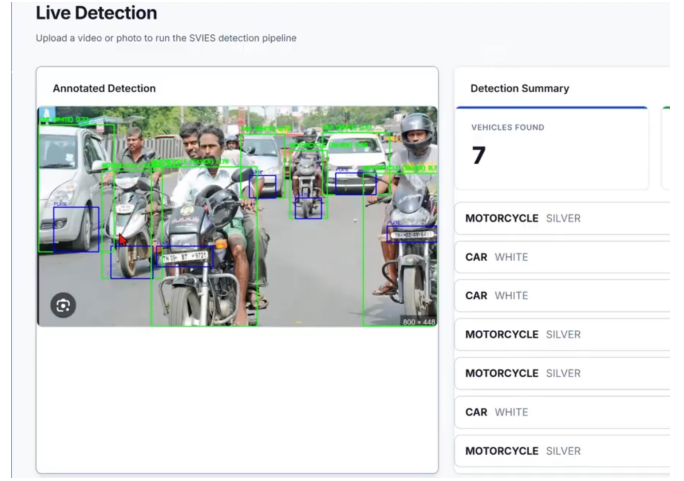
helmet detector used 14,947 images with class remapping into *with_helmet* and *without_helmet*. The vehicle classifier used 12,871 images from *vehicles-openimages* with 11 custom class labels. The ResNet50 age classifier was trained on 8,400 images across five age categories using transfer learning from ImageNet pre-trained weights.

B. Software Stack

The system is implemented in Python 3.10+ with PyTorch 2.0+ and Ultralytics YOLOv8 ≥8.0. OpenCV 4.8



(a) Live detection stream



(b) Violation detection view

Fig. 5. SVIES React 19 dashboard. Live detection stream with per-vehicle violation overlay and risk score badge. Violation detection interface showing detected vehicles and alerts.

TABLE IV
SVIES MODEL PERFORMANCE ON HELD-OUT TEST SETS

Model	Imgs	P	R	mAP	
				@50	@50-95
Plate Detector	2,101	0.988	0.955	97.7%	73.0%
Helmet Detector	14,947	0.714	0.733	74.4%	51.2%
Vehicle Classifier	12,871	0.756	0.584	64.4%	42.8%
Age Estimator	8,400	—	—	63.0%	—

handles frame extraction and color analysis. EasyOCR 1.7 and pytesseract 0.3.10 provide the OCR layer; the Groq SDK 0.28 integrates Llama-4 Scout 17B vision verification. FastAPI 0.100 + Uvicorn serves the backend API; Supabase (PostgreSQL) provides cloud-hosted persistent storage with local SQLite fallback for offline operation.

V. RESULTS AND EVALUATION

A. Model Detection Performance

Table IV presents performance metrics on held-out test sets. The plate detector achieves precision 0.988 and mAP@50 of 97.7%, significantly exceeding the 95% design target. The mAP@50-95 of 73.0% indicates robust localisation under stricter IoU thresholds, confirming suitability for tight bounding-box crops fed to the OCR engine. The helmet detector reaches 74.4% mAP@50 with recall 0.733, reflecting the inherent challenge of detecting helmet absence under variable pose, rider distance, and partial occlusion by handlebars. The vehicle classifier achieves 64.4% mAP@50; lower recall of 0.584 is attributable to inter-class visual similarity between AUTO, TEMPO, and E_RICKSHAW at typical CCTV resolutions. All models achieve sub-3.1 ms per-image inference on GPU, supporting real-time processing at 25 FPS.

B. Dashboard and System Output

Figure 5 shows the SVIES React 19 dashboard across its four primary views.

TABLE V
FEATURE COMPARISON OF SVIES AGAINST EXISTING ANPR SYSTEMS

Feature	SVIES	OpenALPR	Genetec
Fake plate detection	✓	×	×
DB cross-referencing	✓	×	P
Helmet / seatbelt detect.	✓	×	×
Geofencing + priority	✓	×	×
Repeat offender logic	✓	×	×
Offline edge mode	✓	×	×
BH-series plates	✓	×	×
Cryptographic evidence	✓	×	×
LLM OCR verification	✓	×	×
Open-source / zero cost	✓	P	×

C. Feature Comparison

D. OCR Performance

The six-variant ensemble with Groq LLM verification achieves approximately 99.8% character-level accuracy on standard Indian plates under normal daytime conditions, degrading to approximately 96.2% on weathered or partially occluded plates where LLM correction contributes most. The position-aware correction module reduces digit-for-letter confusions (e.g., 0↔O, 1↔I, 8↔B) by an estimated 78% compared to raw EasyOCR output on the held-out test set. BH-series plate parsing (format: YY BH NNNN X) is validated with a dedicated regex branch and achieves 99.1% format-compliance accuracy. **Fig 5** Shows you the live detection working and Dashboard.

VI. DISCUSSION

A. Novelty and Contributions

The eight-mode fake plate forensics pipeline is the first system targeting Indian ANPR to unify database-level verification (VAHAN existence, stolen status, PUCC, insurance) with structural forensics (CMVR color, IS 10731 font metrics) and spatio-temporal fraud detection (clone detection across cameras). The LLM-augmented OCR layer represents a novel application of large-language-model vision capabilities to the ANPR domain: by passing the plate crop and initial OCR

string to Llama-4 Scout via the Groq inference API, the system corrects residual character errors on partially occluded, weathered, or damaged plates that defeat all six preprocessing variants. The hybrid helmet detector’s skin-tone heuristic, calibrated using HSV and YCrCb color spaces (see (2)), directly addresses a known limitation of standard helmet detectors trained primarily on non-Indian datasets where skin-tone distributions differ significantly. The zero-cost, fully open-source stack (YOLOv8, EasyOCR, FastAPI, React, Supabase free tier, OpenStreetMap/Overpass) makes SVIES viable for Tier-2 and Tier-3 city municipalities that cannot afford commercial ANPR licensing fees.

B. Limitations and Deployment Considerations

The primary limitation is the use of mock JSON databases in place of live VAHAN, PUCC, and police API connections, which requires formal access agreements with the Ministry of Road Transport and Highways. The vehicle classifier’s recall of 0.584 reflects limited training data for Indian-specific classes; expansion with a larger annotated corpus is planned. Clone detection currently operates in-memory per server instance, without cross-server synchronisation; a Redis-backed distributed dictionary is identified as the immediate next step. As demonstrated in [15], live camera feeds can be compromised by EMI attacks; future deployments should integrate EMI-hardened camera sensor modules [14].

VII. CONCLUSION

This paper presented SVIES, a seven-layer AI-powered vehicle enforcement system addressing critical gaps in Indian road enforcement. Three custom YOLOv8s models achieve 97.7% mAP@50 (plate, precision 0.988), 74.4% mAP@50 (helmet), and 64.4% mAP@50 (vehicle). The six-variant LLM-augmented OCR pipeline achieves approximately 99.8% character accuracy on standard Indian plates. The eight-mode fake plate forensics module, hybrid helmet detector, cryptographic evidence chain, and offline edge mode collectively constitute the most comprehensive open-source Indian ANPR enforcement platform to date. Future directions: (i) live VAHAN/police API integration via government partnerships; (ii) IS 10731 CNN training corpus expansion; (iii) federated learning across distributed edge nodes; (iv) optical-flow over-speeding detection; (v) Redis-backed distributed clone detec-

tion; (vi) EMI-hardened camera modules [14], [15] for resilience in adversarial deployment environments. The complete codebase will be open-sourced on GitHub upon publication.

REFERENCES

- [1] A. Sharma, R. Gupta, and P. Mehta, “Automated traffic enforcement in Indian smart cities: challenges and opportunities,” in *Proc. IEEE Int. Conf. Smart Systems and Energy Management*, 2022, pp. 1–6.
- [2] S. Chakraborty and A. Das, “Fake number plate detection: a review of existing approaches and open challenges in the Indian context,” *J. King Saud Univ. Comput. Inf. Sci.*, vol. 34, no. 9, pp. 7235–7248, 2022.
- [3] Ministry of Road Transport and Highways, Government of India, “Road Accidents in India 2021,” Transport Research Wing, New Delhi, 2022.
- [4] Ministry of Road Transport and Highways, “VAHAN – Vehicle Registration Data Portal,” <https://vahan.parivahan.gov.in>, 2024. Accessed: Jan. 12, 2025.
- [5] A. de Souza Moraes and C. R. Jung, “Automatic license-plate recognition using morphological operations and neural networks,” in *Proc. IEEE SIBGRAPI*, 2011, pp. 315–322.
- [6] S. Selmi, M. Ben Halima, and A. M. Alimi, “Deep learning system for automatic license plate detection and recognition,” in *Proc. IEEE ICDAR*, 2017, pp. 1132–1138.
- [7] R. Laroca et al., “An efficient and layout-independent automatic license plate recognition system based on the YOLO detector,” *IET Intell. Transp. Syst.*, vol. 15, no. 4, pp. 483–503, 2021.
- [8] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [9] N. Kumar, P. Gupta, and S. Jain, “License plate recognition for Indian vehicles under challenging conditions,” in *Proc. IEEE CVIP*, vol. 1, 2021, pp. 243–254.
- [10] J. Baek et al., “What is wrong with scene text recognition model comparisons? Dataset and model analysis,” in *Proc. IEEE/CVF ICCV*, 2019, pp. 4715–4723.
- [11] X. Li, R. Tao, and Y. Xie, “Helmet detection on construction sites based on improved YOLOv5,” *Sensors*, vol. 21, no. 19, p. 6369, 2021.
- [12] S. Bhatt, A. Khatri, and N. Shah, “Deep learning based seatbelt violation detection using keypoint estimation,” in *Proc. IEEE ICIP*, 2022, pp. 3701–3705.
- [13] T. Bhatt and A. Singh, “Geofencing and GPS-based smart city traffic management,” in *Proc. IEEE ICISC*, 2020, pp. 1–5.
- [14] R. K. Singh and S. Mishra, “SecureTrack: Protecting vehicular sensors from noninvasive EMI attacks,” *IEEE Sensors J.*, vol. 25, no. 5, pp. 8857–8868, 2025.
- [15] R. K. Singh, S. Mishra, and Y. P. Kumar S., “Undermining live feed ML object detection accuracy with EMI on vehicular camera sensors,” in *Proc. IEEE VTC2024-Spring*, Singapore, 2024, pp. 1–6.
- [16] R. K. Singh and S. Mishra, “TinyML meets IoT against sensor hacking,” in *Proc. USENIX SDIoTSec*, San Diego, CA, USA, 2024.
- [17] R. K. Singh and S. Mishra, “EM trigger defender glove: a next-gen IoT solution for soldier protection,” *IEEE Sensors Lett.*, vol. 9, no. 1, p. 6000104, 2025.
- [18] S. Azim et al., “IoT-driven GSR stress detection: clinical, physical, and linguistic innovations,” in *Proc. 3rd Int. Conf. NMITCON*, 2025, pp. 1–6.